Smart FAQ Generator

Abstract

Long documents like company policies, product manuals, and course syllabi contain important information, but users often struggle to find what they need quickly. To address this problem, I built a "Smart FAQ Generator" that automatically converts long documents into clear, helpful FAQ lists. The system extracts text from PDFs, splits it into smaller chunks, and sends each chunk to a large language model (LLM) to generate question–answer pairs.
To understand which LLM and prompt design works best, I ran experiments across three documents and compared two models (OpenAI GPT-4o-mini and Google Gemini 2.0 Flash), using three different prompting styles. I evaluated the outputs using automatic metrics (FAQ count, uniqueness ratio, answer length) and a human rating process covering usefulness, clarity, and correctness.
Across all experiments, **Gemini 2.0 Flash with a structured prompt** consistently produced the highest-quality FAQs. It avoided redundant questions, wrote more complete answers, and achieved perfect scores in human evaluation. I then built a Streamlit web app that lets users upload a PDF, adjust chunk size, view generated FAQs, and search them semantically.
Overall, the system shows that LLMs can make dense documents easier to navigate and understand, and the experimental results highlight how model and prompt choices impact quality.

---

1. Introduction

Many documents contain the information people need, but the way this information is presented is not always user-friendly. Students often skim syllabi searching for deadlines; employees read long policies to find one rule; customers get frustrated flipping through manuals. A common solution is an FAQ page—but writing one by hand takes time and good writing skills.
Large language models are good at extracting and rewriting information, which makes automated FAQ generation worth exploring. The goal of this project is to build a pipeline that takes any long PDF and produces a clean, usable FAQ list.
This project includes:

- A full technical pipeline for FAQ generation
- Experiments comparing different LLMs and prompting approaches
- A Streamlit app that demonstrates the system in a user-friendly way

My main objective was not only to build a working tool, but also to understand what choices (model, prompt, chunking) actually matter in FAQ generation.

---

2. Data

To test the system fairly, I selected three documents that differ in style, structure, and content:

(1) Nikon DSLR User Guide (200 page)

A long and highly technical manual. Full of step-by-step instructions and camera terminology.

(2) Roman Realtors Company Policy (35 pages)

A dense organizational policy document. Rules, procedures, HR sections, and compliance language.

(3) University Course Syllabus (15 pages)

A shorter but more narrative document with dates, grading, and expectations.

Processing Steps

- Text extracted using PyPDF2
- Documents split into chunks (~2,000–2,500 characters each)
- Chunk size adjustable in the Streamlit UI
- All chunks passed through the model independently
- Output collected and deduplicated

This mixture of documents allowed me to test the system on real-world cases

---

3. Experimental Setup

The experiment compared two variables: **LLM provider** and **prompt design**.

Models Tested

- **OpenAI GPT-4o-mini**
- **Google Gemini 2.0 Flash**

Prompt Variants

1. **Simple Prompt** — very minimal instructions
2. **Structured Prompt** — clear format and explicit expectations
3. **User-Centered Prompt** — designed to mimic questions real users might ask

Evaluation Criteria

Automatic Metrics

To analyze the outputs, the system logged:

- Number of generated FAQs
- How many questions were unique vs. duplicates
- Average answer length

These metrics helped understand consistency, verbosity, and repetition.

Human Evaluation

I sampled **90 total Q/As** (30 per document) and manually scored each one based on:

- **Usefulness**
- **Clarity**
- **Correctness**

Scores were averaged for each model + prompt combination.

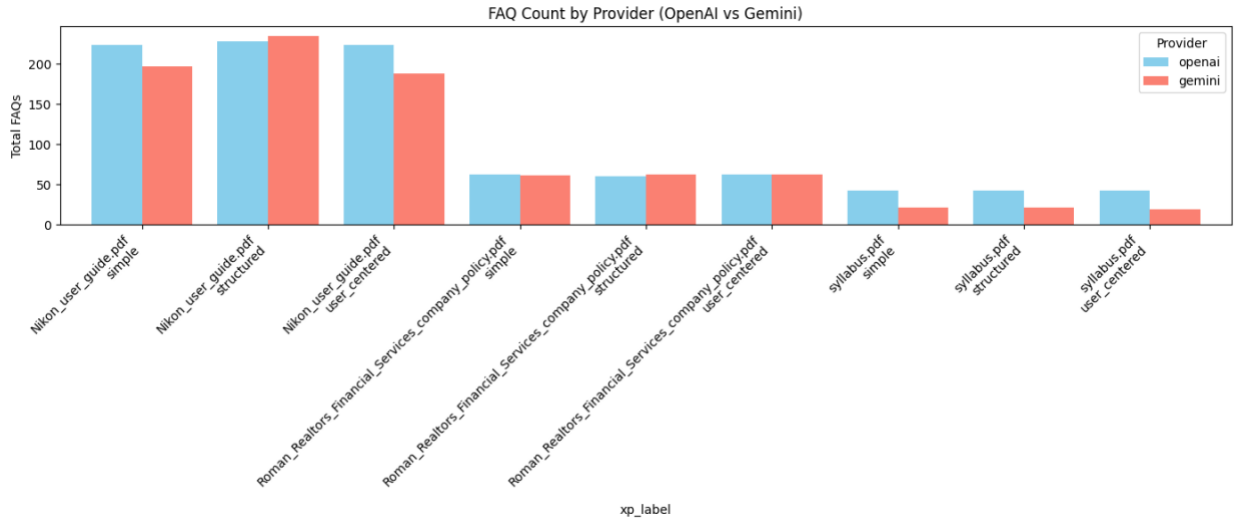
Tools & Libraries

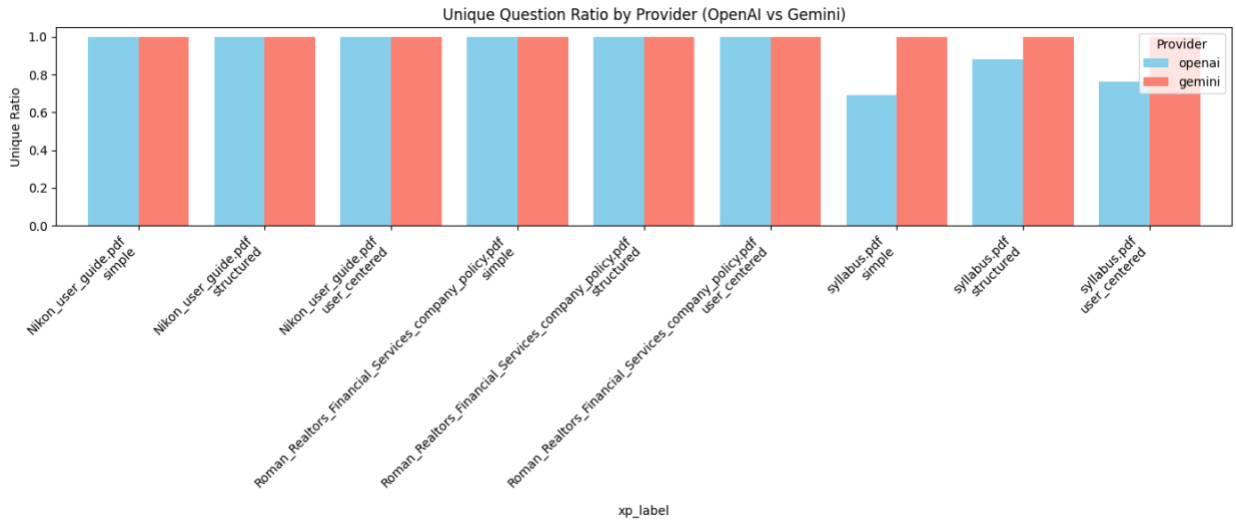
- Python
- OpenAI + Google Generative AI SDKs
- Pandas & Matplotlib
- FAISS for embedding search

- Streamlit for the final demo

---

4. Results





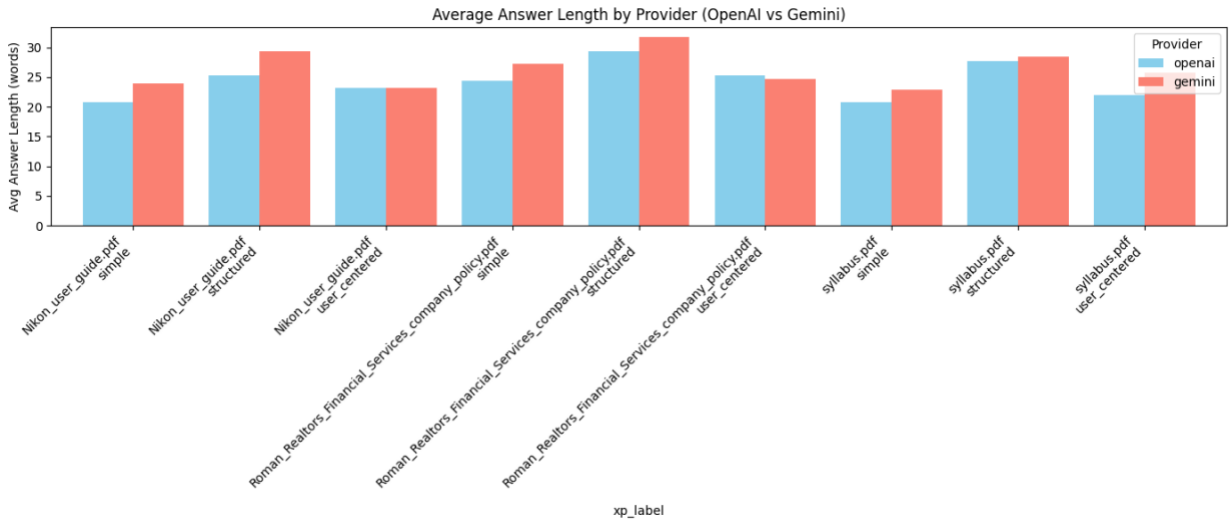


Table 1: Nikon User Guide (200 Pages)

Prompt Style	Gemini Total	Gemini Unique	Gemini Length	Gemini Ratio	OpenAI Total	OpenAI Unique	OpenAI Length	OpenAI Ratio
Simple	197	197	24.01	1.00	224	224	20.82	1.00
Structured	235	235	29.38	1.00	228	228	25.21	1.00
User-Centered	188	188	23.13	1.00	224	224	23.23	1.00

Table 2: Roman Realtors Company Policy (35 Pages)

Prompt Style	Gemini Total	Gemini Unique	Gemini Length	Gemini Ratio	OpenAI Total	OpenAI Unique	OpenAI Length	OpenAI Ratio
Simple	61	61	27.16	1.00	63	63	24.43	1.00
Structured	63	63	31.76	1.00	60	60	29.40	1.00
User-Centered	62	62	24.74	1.00	63	63	25.32	1.00

Table 3: University Course Syllabus (15 Pages)

Prompt Style	Gemini Total	Gemini Unique	Gemini Length	Gemini Ratio	OpenAI Total	OpenAI Unique	OpenAI Length	OpenAI Ratio
Simple	21	21	22.81	1.00	42	29	20.79	0.69
Structured	21	21	28.38	1.00	42	37	27.62	0.88
User-Centered	19	19	25.68	1.00	42	32	22.05	0.76

4.1 Automatic Metrics Overview

Across all three documents:

- **Gemini consistently had the highest unique ratio**, often reaching 1.0. This means it almost never repeated itself, even across large documents.

- **OpenAI generated more total FAQs**, but many were near-duplicates. Gemini produced fewer FAQs because its responses were less repetitive.
- **Answer length:** Gemini's answers were slightly longer and more complete; OpenAI's were shorter but sometimes too short (missing context).

4.2 Human Rating Results

Provider	Prompt	Avg Usefulness	Avg Clarity	Avg Correctness
Gemini 2.0 Flash	**Structured**	**5.0**	**5.0**	**5.0**
OpenAI GPT-4o-mini	Structured	4.9	5.0	4.9
Gemini	Simple	4.9	4.9	5.0
OpenAI	User-Centered	4.8	4.9	4.8

After scoring 90 samples:

- **Gemini 2.0 Flash + Structured Prompt received perfect 5.0 scores across usefulness, clarity, and correctness.**
- OpenAI performed well, but not as consistently—especially on technical documents.

These results matched what I saw qualitatively when reading through the outputs.

---

5. Discussion

Why Gemini 2.0 Flash Performed the Best

After looking at all the metrics and doing the manual scoring, a clear pattern emerged:

4. **Gemini produced the least redundancy**, which dramatically improved the quality of the final FAQ list.
5. Its answers were **grounded** and rarely hallucinated or over-generalized.
6. The **structured prompt** helped shape the output into predictable Q/A pairs without side commentary.
7. Gemini handled all three document types consistently, including the tricky technical manual.
8. It was also **fast**, which matters for the real-time Streamlit app.

OpenAI's model was good, but Gemini produced cleaner, more reliable results.

Effect of Prompting

Prompts made a noticeable difference:

- The **simple prompt** sometimes produced vague questions.
- The **user-centered prompt** occasionally tried too hard to guess what a human might ask.
- The **structured prompt** was the most reliable and was easy for the models to follow.

This combination (Gemini + structured prompt) consistently delivered the best balance of clarity, detail, and accuracy.

---

6. Final System: Streamlit App

To make the project interactive, I built a Streamlit app with:

(1) PDF Upload + FAQ Generation

- Users upload a document.
- Backend runs extraction → chunking → Gemini model → deduplication.
- FAQs appear instantly in a clean list.

(2) Chunk Size Slider

Users can adjust maximum chunk size. This directly impacts:

- number of FAQs
- detail level
- overlap between ideas

This feature shows how important chunking is in LLM workflows.

(3) FAQ Search (Semantic Search)

I added an embedding-based search feature:

- User types a question
- System retrieves the most similar FAQ
- Helps users quickly find relevant answers

This makes the system actually usable, not just a demo.

---

7. Future Directions

There are many ways to extend this system:

- Add multi-document FAQ merging
- Keep track of page numbers and cite sources
- Allow user editing + regeneration for low-quality questions
- Support multilingual FAQ generation
- Use a RAG pipeline for traceability
- Build an admin dashboard showing chunking, model logs, and runtime statistics

---

8. Conclusion

This project shows that LLMs can make dense documents much easier to navigate by automatically generating FAQ pages. Through multiple experiments, I found that **Gemini 2.0 Flash with a structured prompt** produced the highest-quality outputs across different document types. The final Streamlit app demonstrates the full system in a practical, interactive way, including FAQ generation, chunk control, and semantic search.
The approach is flexible, works on real PDFs, and can be applied in many settings—universities, corporations, consumer products, and anywhere people struggle to find information quickly.

-